

Byron's Rules Engine & AI Bot – Integration Guide

Author: Byron Norval (21444758)

Last Updated: 26 September 2025

Contents

1 Quick Start	1
2 Complete Integration Workflows	2
2.1 Human Player Move (WebSocket Handler)	2
2.2 Bot Move (Bot Turn Handler)	3
2.3 Starting a New Game	4
3 Rules Engine API Reference	5
3.1 validateMove(fen, uci)	5
3.2 applyMove(fen, uci)	5
3.3 checkGameStatus(board)	5
3.4 getLegalMoves(fen)	5
4 AI Bot API Reference	5
4.1 generateMove(request)	5
5 Common Mistakes to Avoid & Fixes	6
6 Testing Checklist	7
7 Quick Reference Card	7
8 File Locations & Support	7

1 Quick Start

Critical: Correct Move Processing Flow

Wrong (Common Mistake): Skipping a status check after applying a move can let the game continue after checkmate or stalemate.

Correct Pattern: Always *validate* → *apply* → *check status*.

Listing 1: Engine/Bot imports and construction

```
1 const RulesEngine = require('./src/engine/index.js');
2 const AIBot = require('./src/ai-bot/index.js');
3
4 const engine = new RulesEngine();
5 const bot = new AIBot();
```

Listing 2: Wrong: Missing status check

```
1 const result = engine.applyMove(fen, uci);
2 // MISSING: checkGameStatus() -- game may continue past checkmate
```

Listing 3: Correct: Validate, Apply, Check

```
1 const validation = engine.validateMove(fen, uci);
2 if (!validation.valid) {
3   return { error: validation.error, details: validation.details };
4 }
5 const result = engine.applyMove(fen, uci);
6
7 const board = engine.parseFEN(result.fen);
8 const status = engine.checkGameStatus(board);
9
10 if (status.type === 'CHECKMATE') {
11   // End game -- declare winner
12 } else if (status.type === 'STALEMATE') {
13   // End game -- draw
14 } else if (status.isCheck) {
15   console.log(`${board.turn} is in CHECK!`);
16 }
```

2 Complete Integration Workflows

2.1 Human Player Move (WebSocket Handler)

```
1 async function handlePlayerMove(gameId, playerId, uciMove) {
2   // Get current state (DB)
3   const game = await db.getGame(gameId);
4   const currentFen = game.currentFen;
5
6   // Validate it's their turn
7   if (game.currentTurn !== playerId) {
8     return { error: 'WRONG_TURN' };
9   }
10
11   // 1) Validate
12   const validation = engine.validateMove(currentFen, uciMove);
13   if (!validation.valid) {
14     return { error: validation.error, details: validation.details };
15   }
16
17   // 2) Apply
18   const result = engine.applyMove(currentFen, uciMove);
19
20   // 3) Check status (REQUIRED!)
21   const board = engine.parseFEN(result.fen);
22   const status = engine.checkGameStatus(board);
23
24   // 4) Update DB
25   await db.updateGame(gameId, {
26     currentFen: result.fen,
27     currentTurn: board.turn,
28     gameStatus: status.type,
29     lastMove: uciMove
```

```

30 });
31
32 // 5) Handle end
33 if (status.type === 'CHECKMATE') {
34     const winner = (board.turn === 'white') ? 'black' : 'white';
35     await db.endGame(gameId, { winner, reason: 'checkmate' });
36     return { gameOver: true, winner, newFen: result.fen };
37 }
38 if (status.type === 'STALEMATE') {
39     await db.endGame(gameId, { result: 'draw', reason: 'stalemate' });
40     return { gameOver: true, result: 'draw', newFen: result.fen };
41 }
42
43 // 6) Broadcast
44 broadcastToGame(gameId, {
45     type: 'move_made',
46     move: uciMove,
47     newFen: result.fen,
48     inCheck: status.isCheck,
49     gameStatus: status.type
50 });
51
52 // 7) Trigger bot (if bot game)
53 if (game.opponentType === 'bot') {
54     await handleBotMove(gameId);
55 }
56
57 return { success: true, newFen: result.fen };
58 }

```

2.2 Bot Move (Bot Turn Handler)

```

1 async function handleBotMove(gameId) {
2     const game = await db.getGame(gameId);
3     const currentFen = game.currentFen;
4     const botLevel = game.botLevel; // 'L0' | 'L1' | 'L2' | 'L3'
5
6     // 1) Generate bot move
7     const botResponse = await bot.generateMove({
8         fen: currentFen, level: botLevel, msCap: 5000
9     });
10
11     // 2) Handle bot errors
12     if (!botResponse.ok) {
13         if (botResponse.error === 'NO_LEGAL_MOVES') {
14             const board = engine.parseFEN(currentFen);
15             const status = engine.checkGameStatus(board);
16             await db.endGame(gameId, {
17                 winner: (status.type === 'CHECKMATE') ? 'human' : null,
18                 reason: status.type
19             });
20         }
21         return { error: botResponse.error };
22     }
23
24     // 3) Apply bot move (same pattern as human)
25     const result = engine.applyMove(currentFen, botResponse.move);

```

```

26 // 4) Check status (REQUIRED!)
27 const board = engine.parseFEN(result.fen);
28 const status = engine.checkGameStatus(board);
29
30 // 5) Update DB
31 await db.updateGame(gameId, {
32   currentFen: result.fen,
33   currentTurn: board.turn,
34   gameStatus: status.type,
35   lastMove: botResponse.move
36 });
37
38 // 6) Handle game end
39 if (status.type === 'CHECKMATE') {
40   await db.endGame(gameId, { winner: 'bot', reason: 'checkmate' });
41   return { gameOver: true, winner: 'bot' };
42 }
43 if (status.type === 'STALEMATE') {
44   await db.endGame(gameId, { result: 'draw', reason: 'stalemate' });
45   return { gameOver: true, result: 'draw' };
46 }
47
48 // 7) Broadcast bot move
49 broadcastToGame(gameId, {
50   type: 'bot_move',
51   move: botResponse.move,
52   newFen: result.fen,
53   evaluation: botResponse.evaluation,
54   inCheck: status.isCheck,
55   gameStatus: status.type
56 });
57
58 return { success: true, move: botResponse.move };
59 }
60

```

2.3 Starting a New Game

```

1 async function startNewGame(player1, player2, isBotGame = false, botLevel = 'L1') {
2   const initialFen = 'rnqknr/pppppp/6/6/PPPPPP/RNqKNR w - - 0 1';
3
4   const gameId = await db.createGame({
5     player1,
6     player2: isBotGame ? 'bot' : player2,
7     currentFen: initialFen,
8     currentTurn: 'white',
9     gameStatus: 'ONGOING',
10    botLevel: isBotGame ? botLevel : null
11  });
12
13  return { gameId, initialFen };
14 }

```

3 Rules Engine API Reference

3.1 validateMove(fen, uci)

Checks if a move is legal.

Params fen: string; uci: string

Returns Validation object with valid flag and flags/errors

Listing 4: validateMove return shapes

```
1 { "valid": true, "flags": { "check": false, "capture": false, "promotion": false } }
2 { "valid": false, "error": "ILLEGAL_MOVE", "details": "Piece cannot move that way" }
```

3.2 applyMove(fen, uci)

Applies a move and returns new game state.

Listing 5: applyMove return (example)

```
1 {
2   "fen": "rnqknr/pppppp/6/1P4/6/RNQR b - - 0 1",
3   "status": "ONGOING",
4   "flags": {
5     "check": false,
6     "checkmate": false,
7     "stalemate": false,
8     "threefoldRepetition": false,
9     "fiftyMoveRule": false
10  },
11   "capturedPiece": null
12 }
```

3.3 checkGameStatus(board)

Determines if game is over. Always call after applyMove().

Listing 6: checkGameStatus return (examples)

```
1 { "type": "CHECKMATE", "isCheck": true }
2 { "type": "STALEMATE", "isCheck": false }
3 { "type": "DRAW", "reason": "FIFTY_MOVE", "isCheck": false }
4 { "type": "ONGOING", "isCheck": false }
```

3.4 getLegalMoves(fen)

Returns array of legal UCI moves.

```
1 ["b2b3", "b1c3", "d2d3", /* ... */]
```

4 AI Bot API Reference

4.1 generateMove(request)

Params fen: string; level: 'L0' | 'L1' | 'L2' | 'L3'; msCap (ms, default 5000); seed (optional)

Listing 7: generateMove request

```

1 {
2   "fen": "rnqknr/pppppp/6/6/PPPPPP/RNQKNR w - - 0 1",
3   "level": "L2",
4   "msCap": 5000,
5   "seed": 4242
6 }

```

Listing 8: generateMove success / error

```

1 {
2   "ok": true,
3   "move": "b2b3",
4   "evaluation": 15,
5   "depth": 2,
6   "nodes": 247,
7   "timeMs": 123
8 }
9 {
10  "ok": false,
11  "error": "NO_LEGAL_MOVES",
12  "details": "No legal moves available"
13 }

```

Bot Levels

- L0: Random legal move
- L1: Greedy (prefers captures)
- L2: Minimax (depth 2)
- L3: Enhanced search (depth 3, move ordering, transposition table)

5 Common Mistakes to Avoid & Fixes

1. Not Checking Game Status

```

1 // WRONG: game may continue after checkmate
2 const result = engine.applyMove(fen, uci);
3 return result.fen;

```

```

1 // FIX:
2 const result = engine.applyMove(fen, uci);
3 const board = engine.parseFEN(result.fen);
4 const status = engine.checkGameStatus(board);
5 if (status.type !== 'ONGOING') { /* handle end */ }

```

2. Calling applyMove without validation

```

1 // FIX:
2 const validation = engine.validateMove(fen, uci);
3 if (!validation.valid) return { error: validation.error };
4 const result = engine.applyMove(fen, uci);

```

3. Not Handling Bot Errors

```
1 // FIX:
2 const botMove = await bot.generateMove({ fen, level: 'L2' });
3 if (!botMove.ok) return { error: botMove.error };
4 const result = engine.applyMove(fen, botMove.move);
```

4. Wrong Turn Order

```
1 // Ensure you check whose turn it is before applying a move
```

6 Testing Checklist

Before deploying, test:

1. Normal move sequence (10+ moves)
2. Checkmate detection (game ends correctly)
3. Stalemate detection (draw declared)
4. Check warning (player notified)
5. Invalid move rejection (with proper error)
6. Wrong turn rejection
7. Bot move generation (all 4 levels)
8. Full bot game (start → finish)
9. Promotion handling
10. 50-move rule draw

7 Quick Reference Card

```
1 // 1) VALIDATE
2 const validation = engine.validateMove(fen, uci);
3 if (!validation.valid) return { error: validation.error };
4
5 // 2) APPLY
6 const result = engine.applyMove(fen, uci);
7
8 // 3) CHECK STATUS (REQUIRED!)
9 const board = engine.parseFEN(result.fen);
10 const status = engine.checkGameStatus(board);
11
12 // 4) HANDLE GAME END
13 if (status.type === 'CHECKMATE') { /* end game */ }
14 if (status.type === 'STALEMATE') { /* draw */ }
15 if (status.isCheck) { /* warn player */ }
16
17 // 5) CONTINUE OR BOT MOVE
18 if (status.type === 'ONGOING' && botGame) {
19   const botMove = await bot.generateMove({ fen: result.fen, level });
20 }
```

8 File Locations & Support

- Rules Engine: /backend/src/engine/index.js
- AI Bot: /backend/src/ai-bot/index.js

- Tests: `/backend/unit-tests/`
- Demo: `/backend/cli/main_entry_index.js`

If you encounter issues:

1. Check the error code returned
2. Verify the call order (validate $\rightarrow$ apply $\rightarrow$ status)
3. Ensure status is checked after every move
4. Run the provided unit tests